Matriz de Dependências & Build Times

Modularização, Version Catalogs e Swift Package Manager

Build Engineering

1. GERENCIAMENTO MODERNO DE DEPENDÊNCIAS

A centralização de versões previne inconsistências de transitividade e otimiza as etapas de resolução de dependências em pipelines de CI/CD.

GRADLE VERSION CATALOG (.TOML)

```
[versions]
kotlin = "2.0.0"
retrofit = "2.11.0"

[libraries]
retrofit-core = { module =
"com.squareup.retrofit2:retrofit", version.ref
= "retrofit" }
retrofit-moshi = { module =
"com.squareup.retrofit2:converter-moshi",
version.ref = "retrofit" }

[bundles]
network = ["retrofit-core", "retrofit-moshi"]
```

SWIFT PACKAGE MANAGER (PACKAGE.SWIFT)

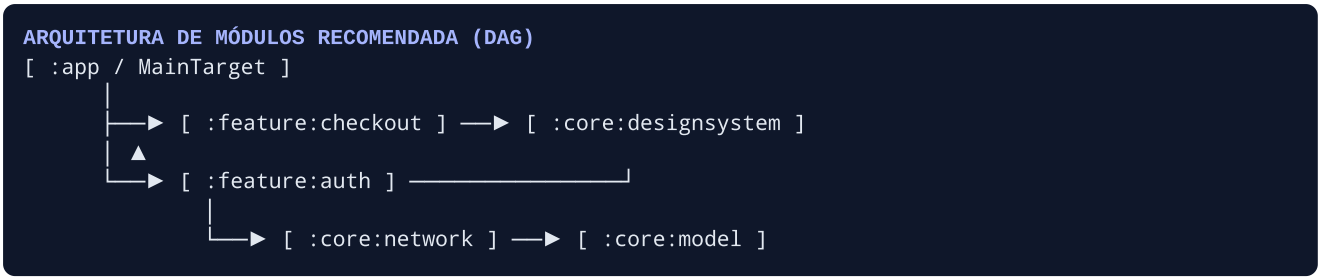
```
// swift-tools-version: 5.10
let package = Package(
  name: "CoreNetwork",
  products: [
    .library(name: "CoreNetwork", targets:
["CoreNetwork"])
  ],
  dependencies: [
    .package(url:
"https://github.com/Alamofire/Alamofire.git",
from: "5.9.0")
  ]
)
```

2. ESTRATÉGIAS DE OTIMIZAÇÃO DE COMPILAÇÃO

Técnica	Android (Gradle)	iOS (Xcode / Swift)
Build Cache	Ativar `org.gradle.caching=true` no `gradle.properties`.	Utilizar `sccache` ou Build Caching via Xcode Cloud.
Modularização	Dividir monolith em módulos `:core`, `:feature` e `:domain`.	Criar Local Swift Packages para quebrar alvos gigantes do Target principal.
Compilação Incr.	Ativar KSP ao invés do legados kapt para processamento de anotações.	Ativar *Whole Module Optimization* apenas para builds de Release.

3. GRAFOS DE DEPENDÊNCIA E PREVENÇÃO DE CICLOS

Módulos devem seguir um fluxo estritamente unidirecional para maximizar a paralelização do build.



4. MÉTRICAS E DIAGNÓSTICO DE BUILD TIMES

Objetivo	Comando / Ferramenta Android	Comando / Ferramenta iOS
Perfilamento de Build	`./gradlew assembleDebug --profile`	Xcode Build Timeline (Product > Perform Action)
Type Checker Slowdowns	Análise de KSP/kapt na aba Build Analyzer.	`-Xfrontend -warn-long-expression-type-checking=100`
Análise de Grafos	Plugin `com.jraska.module-graph-assert`	`swift package show-dependencies`

5. BOAS PRÁTICAS ABSOLUTAS EM CI/CD

- **Nunca faça clean build desnecessário no CI:** Reutilize caches de dependências entre execuções de Pull Requests.
- **Mantenha Toolchains Atualizadas:** Atualizações do Kotlin (2.x com K2 compiler) e Swift 6 trazem melhorias expressivas de velocidade de compilação.
- **Evite acoplamento de Módulos de Feature:** O módulo `:feature:checkout` NUNCA deve importar o módulo `:feature:auth` diretamente. Utilize navegação orientada a URLs/DeepLinks ou interfaces de navegação expostas no `:core`.